

Python is considered the most important language in AI & ML for several key reasons:-

1) Ease of Learning and use.

- Python has a simple and readable syntax, making it easier for developers and researchers to focus on solving AI/ML problems rather than struggling with complex programming concepts.

2) Extensive Libraries and Frameworks.

- Python provides a vast collection of pre-built libraries that make AI/ML development more efficient:-

Numpy, SciPy and Pandas - for data manipulation and scientific computing

Matplotlib and Seaborn - for data visualization

Scikit-learn - for machine learning algorithms.

TensorFlow and PyTorch - for deep learning and neural networks

NLTK and SpaCy - for natural language processing.

3) Strong Community Support:-

- Python has one of the largest and most active developer communities, meaning constant updates, troubleshooting help, and shared open-source projects

4) Platform Independence

- Python runs on different platforms (like OS) without major modifications, making it highly flexible for various AI/ML applications.

- 5) Integration with other Technologies
 - Python can easily integrate with C, C++, Java, and other languages, allowing developers to optimize performance-heavy tasks when needed.
- 6) Support for Big Data and cloud Computing
 - Python is widely used in big data analytics and supports cloud-based solutions for AI/ML development on platforms like AWS, Google Cloud, and Azure.
- 7) Flexibility and Rapid prototyping
 - Python enables fast prototyping of AI/ML models, allowing researchers to quickly test and iterate ideas.
- 8) Widespread Industry Adoption
 - From startups to tech giants like Google, Microsoft, and Facebook, Python is the preferred language for AI & ML development.

Wap to take input sequence from user and sort in Ascending order.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int n, i, j, temp;
```

```
printf("Enter the no of elements:");
```

```
scanf("%d", &n);
```

```
int arr[n];
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
scanf("%d", &arr[i]);
```

```
}
```

```
for (i = 0; i < n - 1; i++)
```

```
{
```

```
for (j = 0; j < n - i - 1; j++)
```

```
{
```

```
if (arr[j] > arr[j+1])
```

```
{
```

```
temp = arr[j];
```

```
arr[j] = arr[j+1];
```

```
arr[j+1] = temp;
```

```
}
```

```
printf("Sorted in Ascending order: \n");
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
printf("%d\t", arr[i]);
```

```
}
```

```
printf("\n");
```

```
}
```


Bubble Sort for Descending order:-

$[5, 2, 9, 1, 6]$

Step-by-step process:-

Pass 1:

- Compare $5 \times 2 \rightarrow$ No swap
- Compare $2 \times 9 \rightarrow$ swap $\rightarrow [5, 9, 2, 1, 6]$
- Compare $2 \times 1 \rightarrow$ No swap
- Compare $1 \times 6 \rightarrow$ swap $\rightarrow [5, 9, 2, 6, 1]$

Pass 2:

- Compare $5 \times 9 \rightarrow$ swap $\rightarrow [9, 5, 2, 6, 1]$
- Compare $5 \times 2 \rightarrow$ No swap
- Compare $2 \times 6 \rightarrow$ swap $\rightarrow [9, 5, 6, 2, 1]$

Pass 3:

- Compare $9 \times 5 \rightarrow$ No swap
- Compare $5 \times 6 \rightarrow$ swap $\rightarrow [9, 6, 5, 2, 1]$

Pass 4:

- Compare $9 \times 6 \rightarrow$ No swap

Final Sorted array (descending order)

$[9, 6, 5, 2, 1]$

Comparison of Sorting Algorithms:

Algorithm	Best case	worst case	Average case	Space complexity	Stable?
✓ Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
✓ Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗
✓ Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	—
Quick sort	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(\log n)$	✗
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗
Counting sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	✓
Radix sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	✓

* Bubble sort:

How it works:-

- Repeatedly swaps adjacent element if they are in the wrong order
- Largest element "bubble" to the top after each pass

Pros:

- Simple to implement
- Stable sorting (preserve order of equal elements)

cons:

- Slow for large datasets ($O(n^2)$)
- Too many unnecessary swaps

Best for:

- Small arrays or nearly sorted data.

Idea:-

- Repeatedly swap adjacent elements if they are in the wrong order
- Largest elements 'bubble up' to the last position

Pseudo-code for Ascending order:

Bubble sort(A, n)

for $i = 0$ to $n-1$

for $j = 0$ to $n-i-1$

if $A[j] > A[j+1]$

swap($A[j]$, $A[j+1]$)

Ex: arr:

[9 2 5 1]

pass 1:

compare 9 & 2 $\rightarrow$ swap [2 9 5 1]

compare 9 & 5 $\rightarrow$ swap [2 5 9 1]

compare 9 & 1 $\rightarrow$ swap [2 5 1 9]

Pass 2

compare 2 & 5 $\rightarrow$ No swap

compare 5 & 1 $\rightarrow$ swap [2 1 5 9]

Pass 3

compare 2 & 1 $\rightarrow$ swap [1 2 5 9]

Final [1 2 5 9]

Advantages of bubble sort over selection and insertion sort.

1) Stable Sorting Algorithm

- preserves the order of duplicate element, unlike selection sort.

2) Easy to Implement

- Simple and requires fewer lines of code

Disadvantages

- very slow for large datasets ($O(n^2)$ complexity)
- more swaps than selection sort, making it inefficient for high swap-cost situations.

② Selection Sort:

How it works:

- Finds the smallest element and swap it with the first element, then repeats for the remaining elements.

Pros

- Fewer swaps than bubble sort
- works well when swapping is costly

cons:

- Always $O(n^2)$, even if the array is sorted.
- Not a stable sort.
- Best for:
 - when swapping operations are expensive (e.g. memory writes)

Idea

- Find the smallest element in the unsorted part.
- Swap it with the first element of the unsorted part.

Pseudo-code

selection sort (A, n)

for $i = 0$ to $n-1$

$minindex = i$

 for $j = i+1$ to $n-1$

 if $A[j] < A[minindex]$

$minindex = j$

 swap ($A[i], A[minindex]$)

Day - 24/11

[5, 2, 9, 1, 6]

Pass 1:

Find smallest = 1 swap 1 with 5 $\rightarrow$ [1, 2, 9, 5, 6]

Pass 2:

Find smallest in [2, 9, 5, 6] $\rightarrow$ No swap

Pass 3:

Find smallest = 5 swap 5 with 9 $\rightarrow$ [1, 2, 5, 9, 6]

Pass 4

Find smallest = 6 swap [9, 6] $\rightarrow$ [1, 2, 5, 6, 9]

Hence

sorted = [1 2 5 6 9]

Advantages:

- Fewer swap than bubble and insertion sort ($O(n^2)$)
- Easy to implement and requires no extra space.

Disadvantages

- No adaptive (always runs in $O(n^2)$ even if the array is nearly sorted)
- Not stable (Doesn't maintain the relative order of equal elements)

③ Insertion Sort.

- How it works
- Takes each element and inserts it into its correct position in the sorted part of the array.
- Pros
 - Fastest of the three when data is nearly sorted
 - Stable sorting, works well for small datasets.

Cons

- Slow for large, random datasets ($O(n^2)$)

Best for

- small datasets
- Nearly sorted arrays

Idea

Take each element and insert it into the correct position in the sorted part.

Pseudo-code:

InsertionSort (A, n)

for $i = 1$ to $n-1$

key = $A[i]$

$j = i - 1$

while $j > 0$ and $A[j] > \text{key}$

$A[j+1] = A[j]$

$j = j - 1$

$A[j+1] = \text{key}$

Day 200:

[5, 2, 9, 1, 6]

Pass 1:

move 5 to the right $\rightarrow$ [5, 5, 9, 1, 6]

insert 2 at position 0 $\rightarrow$ [2, 5, 9, 1, 6]

Pass 2:

insert 9 at correct place $\rightarrow$ No change

Pass 3:

move 9 $\rightarrow$ [2, 5, 9, 9, 6]

move 5 $\rightarrow$ [2, 5, 5, 9, 6]

move 2 $\rightarrow$ [2, 2, 5, 9, 6]

insert 1 $\rightarrow$ [1, 2, 5, 9, 6]

Pass 4:

move 9 $\rightarrow$ [1, 2, 5, 9, 9]

insert 6 at position 3 $\rightarrow$ [1, 2, 5, 6, 9]

Done

Advantages:

- Efficient for small or nearly sorted data ($O(n)$ best case)
- Stable sorting algorithm
- Adaptive

Disadvantages:

- Performs poorly on large datasets ($O(n^2)$ worst case)
- more swaps than selection sort in the worst case

Find the factorial of a number using a recursion function:

```
def factorial (n):
```

```
    if n == 0
```

```
        return 1
```

```
    return
```

```
    else:
```

```
        return n * factorial (n-1)
```

```
n = int(input("Enter a number"))
```

```
print ("Factorial of", n, "is", factorial (n))
```

```
dry run:
```

fact0	0 * fact(1)	5 * 4 * 3 * 2 * 1 * 1
fact1	1 * fact(0)	5 * 4 * 3 * 2 * 1 * fact(0)
fact2	2 * fact(1)	5 * 4 * 3 * 2 * fact(1)
fact3	3 * fact(2)	5 * 4 * 3 * fact(2)
fact4	4 * fact(3)	5 * 4 * fact(3)
fact5	5 * fact(4)	5 * fact(4)

To find the n^{th} term of a Fibonacci series using recursion.

```
def fibonacci(n):
```

```
    if n <= 0:
```

```
        return "Invalid input"
```

```
    elif n == 1:
```

```
        return 0 # First
```

```
    elif n == 2:
```

```
        return 1 # Second
```

```
    else
```

```
        return fibonacci(n-1) + fibonacci(n-2)
```

```
num = int(input("Enter the term of Fibonacci series to print"))
```

```
print(f"The {num}th Fibonacci number is {fibonacci(num)}")
```

Dry run: (6)

fibonacci(6)

→ fibonacci(5)

→ fibonacci(4)

→ fibonacci(3)

→ fibonacci(2) = 1

→ fibonacci(1) = 0

→ fibonacci(2) = 1

→ fibonacci(1) = 0

→ fibonacci(3)

→ fibonacci(2) = 1

→ fibonacci(1) = 0

→ fibonacci(4)

→ fibonacci(3)

→ fibonacci(2) = 1 + fibonacci(1) = 0

→ fibonacci(2) → ~~fibonacci(1) = 0~~

→ = 1

Sum of all digits of a number 'n' using recursion.

```
def sum_of_digits(n):
```

```
    if n == 0:
```

```
        return 0:
```

```
    return (n % 10) + sum_of_digits(n // 10) # add + call
```

```
num = int(input("Enter a number"))
```

```
print(f"Sum of digits of {num} is {sum_of_digits(num)}")
```

dry-run (1234)

sum_of_digits(1234)

└─ 4 + sum_of_digits(123)

└─ 3 + sum_of_digits(12)

└─ 2 + sum_of_digits(1)

└─ 1 + sum_of_digits(0)

└─ 0 (return)

Ans

$$4 + 3 + 2 + 1 = 10$$

Reverse a string using a recursion.

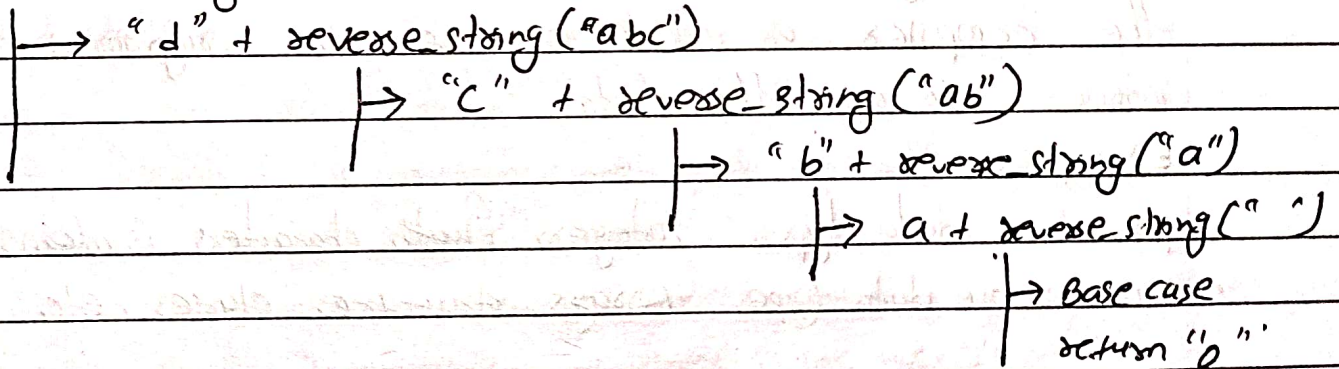
```
def reverse_string(s):
    if len(s) == 0: #
        return ""
    elif
        return s[-1] + reverse_string(s[:-1])
```

Last character + reverse of remaining string

```
string = input("Enter a string:")
print(f"Reversed string: {reverse_string(string)}")
```

dry-run

reverse_string("abcd")



1K

Introduction to Data Structures.

What are Data Structure?

- A data structure is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently. It provides a means to manage large amounts of data, such as database or internet indexing services.

Examples:- Arrays, Linked lists, Stacks, Queues, Tree, Graphs. etc.

Data Types:-

A data type is an attribute of data that tells the compiler or interpreter how the programmer intends to use the data.

Examples:-

- primitive data types: integers, floats, characters, Booleans
- Non-primitive data types: Arrays, structures, classes, etc.

Efficient use of memory

- Efficient memory usage involves optimizing the storage and retrieval of data to minimize memory consumption and maximize performance.

Examples:

using an array instead of multiple variables to store a list of numbers.

Recursion

Recursion is a programming technique where a function calls itself in order to solve a problem. The function must have a base case to terminate the recursion.

Example:-

```
def factorial(n):
    if n == 0:
        else: return 1
    else:
        return n * factorial(n-1)
```

Explanation: The factorial function calls itself with a decremented value of n until it reaches the base case ($n = 0$).

Q

- What do you mean by recursion
- What is the importance of recursion

Time complexity of recursion and base condition in a recursion

→ The time complexity of recursion depends on recurrence relation, how the function calls itself.

Common case: $T(n) = T(n-1) * n$ (eg factorial), we master theorem to solve recurrence relations.

Base condition in Recursion

Prevents infinite recursion and define stopping criteria

eg: find Fact(n)?

```
if (n == 0 || n == 1) return 1; // Base case
return n * fact(n-1);
```

→ Without base condition, recursion leads to stack overflow

Find the Best, worst and Average case scenarios

1) Bubble Sort:

- Best case ($O(n)$): when the array is already sorted only one pass is needed with no swaps.
- worst case ($O(n^2)$): when the array is in reverse order, requiring maximum swaps.
- Average case ($O(n^2)$): In general, bubble sort compares each element with others leading to quadratic complexity.

dry-run:

- Best case ($O(n)$) → Already sorted
[1, 2, 3, 4, 5] // only one pass, no swap
- worst case ($O(n^2)$) → Reverse sorted:
[5, 4, 3, 2, 1] // max swaps needed
- Average case ($O(n^2)$) → Random order
[3, 1, 4, 2, 5] // some swaps needed

2) Insertion Sort:

- Best case ($O(n)$): if the array is already sorted, it only requires $n-1$ comparisons and no swaps.
- Worst case ($O(n^2)$): if the array is in reverse order, each element needs to be moved to the correct position, leading to quadratic time.
- Average case ($O(n^2)$): on average, elements need to be shifted, leading to $O(n^2)$ complexity.

dry-run:

- Best case ($O(n)$) $\rightarrow$ Already sorted:
[1, 2, 3, 4, 5] // no shifts needed
- Worst case ($O(n^2)$) $\rightarrow$ Reverse sorted:
[5, 4, 3, 2, 1] // each element moves left fully
- Average case ($O(n^2)$) $\rightarrow$ Random order:
[3, 2, 1, 4, 5] // some shifts needed

3) Binary Search:

Best case ($O(1)$): if the element is found at the middle index in the first comparison.

Worst case ($O(\log n)$): if the element is at the first or last position, requiring the max no of divisions.

Average case ($O(\log n)$): in general, Binary search discards half of the array in each step, leading to logarithmic complexity.

dry-run:

- Best case ($O(1)$): Found in the first check.
Search 5 in $[1, 3, 5, 7, 9]$ // mid = 5 found instantly
- Worst case ($O(\log n)$) $\rightarrow$ Last position found after $\log n$ steps
Search 1 in $[1, 3, 5, 7, 9]$ // need $\log n$ comparisons.
- Average case ($O(\log n)$) $\rightarrow$ middle element not found immediately but in a few steps

Search 4 in $[1, 3, 5, 7, 9]$ // not found, 2-3 steps needed

- Why do we consider worst case scenario as a measuring factor in programming?

→ We consider the worst-case scenario in programming because

- 1) Guarantees performance → Ensure the algorithm performs within acceptable limits, even in the most difficult cases.
- 2) Avoids unexpected delays → prevents slow execution in edge cases, improving reliability.
- 3) Standardized comparison → Helps compare algorithms fairly by analysing their maximum possible execution time.
- 4) Real-world impact → critical for time-sensitive applications e.g. database.

Examples:

- Quick sort (worst case): $O(n^2)$ if the pivot is always the smallest/largest.
- Merge sort (worst-case): $O(n \log n)$ guaranteed performance

Helps to choose most-efficient and reliable algorithm for real world applications

- What is space complexity and what is its importance

→ space complexity is a measure of how much extra memory (RAM) a program requires during execution, including variables, function calls, and data structure.

Why is it important?

- effective memory usage
- optimization in large data processing
- Recursion vs Iteration
- embedded systems & microcontrollers
- multi-tasking & performance

Difference between time and space complexity

Time complexity

- measures the execution time of an algorithm as input size grows.

- no. of operations performed

Space complexity

- measures the memory used by an algorithm including input, variable and recursion

- Amount of memory allocated

- What do you mean by Asymptotic notation. What is its importance in context of programming

→ Asymptotic notation is mathematical way to describe the time complexity and space complexity of an algorithm as the input size(n) grows. It helps us to analyze how an algorithm performs for large inputs without focusing on machine specific details.

Importance:

- i) performance Analysis
- ii) Scalability
- iii) Ignore constant factors
- iv) optimization.

Star

```

      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * *
* * * * *

```

Ajanta

Page No. _____

Date _____

```

for (int i=1; i<=row; i++)

```

```

{
    for (int j=1; j<=row-i; j++)

```

```

    {
        print (" ");

```

```

    }

```

```

    for (int k=1; k<=i; k++)

```

```

        {
            print ("*");

```

```

        }

```

```

    print ("\n");

```

```

}

```

row = 5

for i in range (1, row+1):

print (" " * (row-i) + "*" * i)

```

for (i = row; i >= 1; i--)

```

```

{
    for (int j=0; j<=row-i; j++)

```

```

    {
        print (" ");

```

```

    }

```

```

    for (int k=0; k<=i; k++)

```

```

    {
        print ("*");

```

```

    }

```

```

    print ("\n");

```

row = 5

for i in range (row, 0, -1):

print (" " * (row-i) + "*" * i)

1.6) Big O Notation and Theta Notations.

→ Big O Notation: Describes the upper bound of an algorithm's running time. It represents the worst-case scenario.

Example:

$O(n)$ means the algorithm's running time grows linearly with the input size.

Theta Notation (Θ): Describes the average-case running time of an algorithm.

Example:

$\Theta(n)$ means the algorithm's running time grows linearly with the input size in the average case.

2. Elementary Data Structures:-

2.1) Stacks:-

- A stack is a linear data structure that follows the Last in First out (LIFO) or First in Last out (FILO) principle.

The last element added to the stack will be the first one to be removed.

Operations:

Push (append) : Add an element to the top of the stack

pop : Remove the element from the top of the stack.

peek/top : view the top element without removing it.

Example

Stack = []

Stack.append(1) # Add

Stack.append(2)

Stack.append(10)

Stack.pop() # remove

Stack.append(3)

Stack.append(10)

Stack.append(2)

Stack.pop()

Stack.append(10)

Stack.pop()

print(Stack)

print(Stack[-1]) # top

output

[1, 2, 3, 10]

10

points to be noted

- 1) Lifo $\rightarrow$ principle
- 2) Basic operation: push(), pop(), peek(), is_empty()
- 3) Stack (overflow), (underflow) $\rightarrow$ unwanted pop
- 4) Applications
 - $\hookrightarrow$ Function call stack
 - $\hookrightarrow$ undo/redo in text editors
 - $\hookrightarrow$ expression evaluation
 - $\hookrightarrow$ Backtracking algorithms.

- Implement Stack using a n size array using python

Stack []

Stack.append(10)

Stack.append(20)

Stack.append(30)

print(Stack, stack)

10, 20, 30

print("Popped:", stack.pop())

30

print("Popped:", stack.pop())

20

print("Top element:", stack[-1])

10

print("Popped:", stack.pop())

10

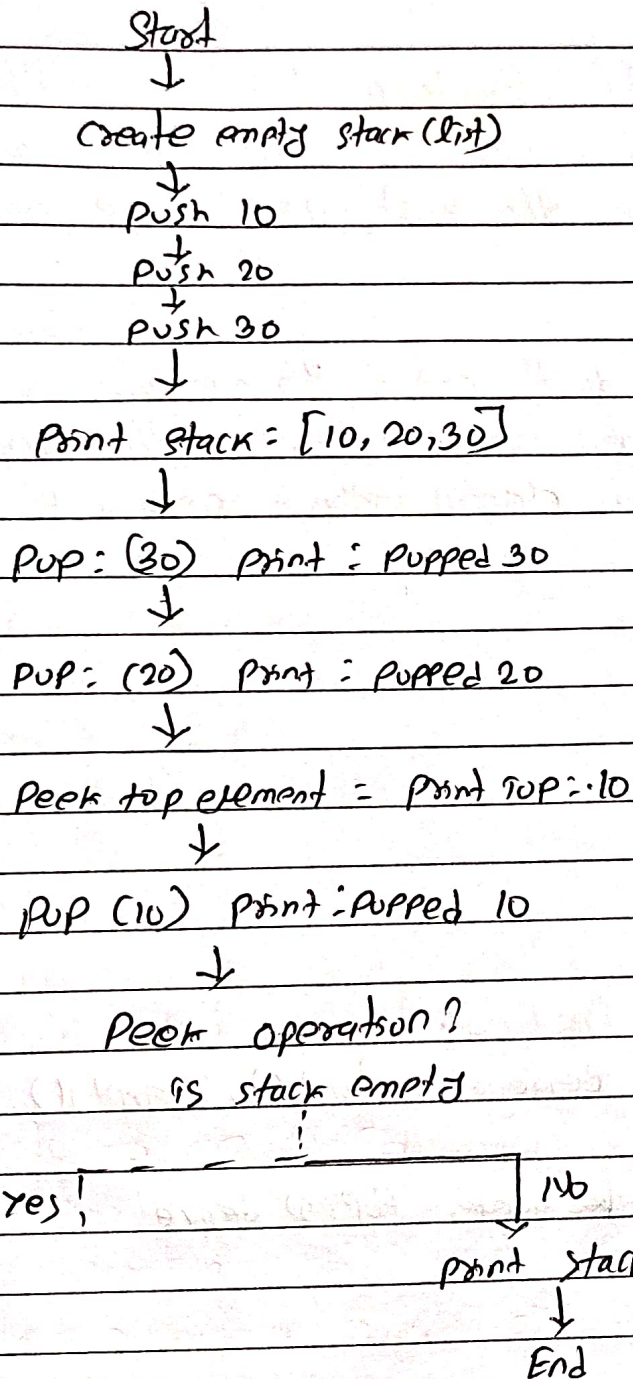
if stack:

print("Top element :", stack[-1])

else:

print("Stack is empty")

Day - 24



22) Queues:

- A queue is a linear data structure that follows the first in first out (FIFO) principle. The first element added to the queues will be the first one to be removed.

Operations:

Enqueue: Add an element to the end of the queue.

Dequeue: Remove the element from the front of the queue.

peek/front: view the front element without removing it.

Ex:-

Queue = deque()

Queue.append(1)

Queue.append(2)

Queue.popleft()

Points to be noted:

- 1) FIFO principle $\rightarrow$ First in, First out
- 2) Basic operation = enqueue(x), dequeue(), front(), is empty()
- 3) Implementation
 - $\rightarrow$ List $\rightarrow$ append() for queue, pop() deque
 - $\rightarrow$ collection.deque
 - $\rightarrow$ Linked list
- 4) Applications of queues
 - $\rightarrow$ CPU scheduling, Print queue
 - $\rightarrow$ Breadth-first search, Call center system.

Program

size = 5

queue = [None] * size

front = -1

rear = -1

// enqueue

rear = rear + 1

if front == -1:

front = 0

queue[rear] = 10

rear = rear + 1

queue[rear] = 20

rear = rear + 1

queue[rear] = 30

print("Queue:", queue[front:rear+1])

// dequeue

if front > rear or front == -1:

print("Queue underflow! cannot deque")

else:

print("Dequeued:", queue[front])

front = front + 1

if front > rear or front == -1:

print("Queue is empty")

else:

print("front element:", queue[front])

print("Queue after deque:", queue[front:rear+1])

DoJ-runStart

Define queue / size = 5

Rount = -1 & rear = -1



enqueue = 10

Rear = 0, Rount = 0



enqueue = 20

Rear = 1



enqueue = 30

Rear = 2



Print queue

Output = [10, 20, 30]



Dequeue operation

Dequeue = 10

Rount = 1



Peek Rount element

Output = 20



Print queue again

Output = [20, 30]



end

Questions
25/02/2025

Ajanta

Page No. _____

Date _____

Difference between linked list and array in context of memory utilization.

→ The difference are as follows:-

Feature	Array	Linked list
Memory allocation	contiguous (fixed-size)	Non-contiguous (dynamic)
Memory Efficiency	can waste memory if size is too large	uses only required memory
Wastage	may have unused memory (if not null)	No unused memory (except pointers)
Extra overhead	No extra space needed	Extra memory needed for pointers
Access time	fast ($O(1)$) via indexing	slower ($O(n)$ traversal needed)
Resizing	Requires copying to a new array	No need for resizing grows dynamically

Key points:

- i) Array require contiguous memory, while linked list use scattered memory blocks.
- ii) Arrays may have unused memory, but linked list use extra space for pointers
- iii) Arrays provide fast access ($O(1)$) while linked list require traversal ($O(n)$)
- iv) Linked list are better for dynamic memory allocation, while arrays are better for fast indexing.

② How linked list performs better to a array when it comes to , deletion and insertion operation.

→ There are some points which describe it better

i) No shifting required:

- In an array, inserting or deleting an element requires shifting all element after it ($O(n)$).
- In a linked list, we only update pointers ($O(1)$) if we have the node reference.

ii) Efficient Dynamic size

- Array: fixed or resized over data, less data (Array size)
- Linked list: just updates pointers, making it ($O(1)$) grows and shrinks dynamically without extra cost.

iii) Efficient middle insertion & Deletion.

- Array: must shift elements
- Linked list: just update pointers, making it ($O(1)$) (if node reference is given)

Array are better in following ways:-

- Fast Random access $O(1)$ via indexing
- Less memory overhead (No extra pointer storage like linked list).

③ Why an array still performs better when it comes to retrieval operation.

→ Some of the reasons why array still performs better are as:-

- i) In an array, elements are stored contiguously in memory, this allows instant access using array [index].
- ii) Fast Random access (O(1)) via indexing
- iii) Less memory overhead, No extra pointer storage like linked list
- iv) They have better cache locality since elements are stored sequentially in memory.

④ What do you mean by linked list in Data structure. Give some real examples.

→ A linked list is dynamic data structure whose elements (nodes) are stored in non-contiguous memory locations. Each node consists of:-

- i) Data (the actual value)
- ii) Pointer (next) (a reference to the next node in the sequence)

Examples:-

- i) Train coaches
- ii) music Player
- iii) social media feeds
- iv) undo / redo function

Note: linked list are → Dynamic size: no need to define fixed size.

→ Fast insertion & deletion

→ Efficient for large data: No memory wastage due to resize.